Handgelus · Documento de arquitectura

1. Introducción

Handgelus es una aplicación web que centraliza la operación del atelier Handgelus (Armenia, Quindío), dedicado al tatuaje de autor y al arte por comisión. Sustituye la gestión manual por redes sociales y notas en papel por un solo sistema. En él, el cliente solicita su servicio, recibe una cotización, paga el abono o anticipo y sigue el avance de su pieza. El equipo, por su parte, cotiza, agenda y registra la producción.

El sistema está construido con un backend Java 21 / Spring Boot 3.5 organizado en **arquitectura hexagonal (puertos y adaptadores)**, una base de datos **Oracle** y un frontend **Angular 19**.

Objetivos del informe

- Documentar la arquitectura de Handgelus con el modelo **C4** (contexto, contenedores, componentes y código) y la vista **+1** de escenarios mediante diagramas de secuencia.
- Explicar las responsabilidades de cada componente y cómo se relacionan.
- Justificar las decisiones de diseño y los patrones aplicados.
- Describir la estrategia de calidad, el despliegue y las medidas de seguridad.
- Servir de referencia para el mantenimiento y la evolución del sistema en el curso de Ingeniería de Software II.

2. Contexto del sistema

Dominio del problema

El atelier ofrece dos servicios: **tatuaje de autor**, que se realiza en una o varias sesiones en el estudio, y **obra por comisión**, una ilustración u óleo que se envía a la dirección del cliente. Hasta ahora, las solicitudes llegaban por redes sociales, las cotizaciones se calculaban a mano, los abonos se anotaban en papel y la agenda era informal. El resultado eran información perdida, dobles citas y nula trazabilidad del avance de las obras.

Los actores son cuatro: el **cliente**, la **asistente** (bandeja, agenda y pagos), el **tatuador/artista**, que también es el propietario (cotiza, configura tarifas y produce) y el **administrador** (cuentas y roles).

Requerimientos funcionales

ID	Funcionalidad	Requisitos de software clave
F-01	Gestionar usuarios del sistema	SWR-01 rol único por cuenta · SWR-02 acceso por rol · SWR-03 bcrypt y HTTPS
F-02	Configurar tabla de tarifas	SWR-04 tarifas por tamaño y servicio · SWR-05 los cambios no alteran cotizaciones emitidas
F-03	Registrar solicitud de servicio	SWR-06 correo único · SWR-07 imagen ⩽ 5 MB · SWR-08 dirección completa en comisiones
F-04	Generar cotización	SWR-09 cálculo automático · SWR-10 solo el tatuador ajusta y confirma · SWR-11 notificación y decisión del cliente
F-05	Registrar pago de abono o anticipo	SWR-12 mínimo 30 % / 50 % · SWR-13 verificación por la pasarela · SWR-14 reversión a las 48 h
F-06	Agendar cita de tatuaje	SWR-15 máximo 2 citas por día · SWR-16 comprobante · SWR-17 sin abono no hay cita
F-07	Actualizar avance de obra	SWR-18 estados de la obra · SWR-19 el cliente solo ve lo suyo · SWR-20 no se envía con saldo
F-08	Consultar historial de servicios	SWR-21 tipo, fecha, valor, estado y comprobantes

Reglas de negocio y restricciones

- **RN-01 / RN-02 / RN-10:** sin abono del 30 % no hay cita; las comisiones inician producción solo con el 50 % de anticipo.
- **RN-03 / RN-04:** los tatuajes pequeños tienen tarifa fija; los trabajos de más de 4 horas se dividen en sesiones.
- **RN-05:** dos jornadas por día (mañana y tarde), una cita por jornada; domingos sin atención.
- **RN-07 / RN-11:** un servicio se cierra, o una obra se envía, solo con el pago total.
- **RN-09:** el envío de comisiones cuesta $12.000 en Armenia y $28.000 en el resto del país.
- **Restricciones técnicas:** Java 21 con Spring, Angular, Gradle, configuración por *properties*, base de datos Oracle e IntelliJ IDEA como IDE. La pasarela de pago y el correo son simulados en el entorno académico. El tratamiento de datos se rige por la Ley 1581 de 2012.

Tecnologías utilizadas

Capa	Tecnología	Uso
Lenguaje backend	Java 21	Records, *switch* de expresiones, *pattern matching*
Framework backend	Spring Boot 3.5 (Web, Validation, Data JPA, Security, AOP)	API REST, persistencia, seguridad, transacciones
Seguridad	Spring Security, JJWT 0.12, BCrypt (factor 12)	Autenticación sin estado y contraseñas cifradas
Base de datos	Oracle XE 21c / 23ai Free (ojdbc11); H2 en modo Oracle como perfil opcional	Persistencia de las 10 tablas `HG_*`
Construcción	Gradle 8.14 (wrapper), multiproyecto	Compilación, pruebas y ejecución
Frontend	Angular 19 (componentes *standalone*, *signals*), TypeScript 5.7, RxJS 7.8, SCSS	Portal del cliente y panel del atelier
Contenedores	Docker Compose con la imagen `gvenzl/oracle-free:23-slim`	Oracle local para desarrollo
Pruebas	JUnit 5, Spring Boot Test	Prueba de flujo de negocio
IDE	IntelliJ IDEA	Configuraciones de ejecución en `.run/`

3. Arquitectura de alto nivel

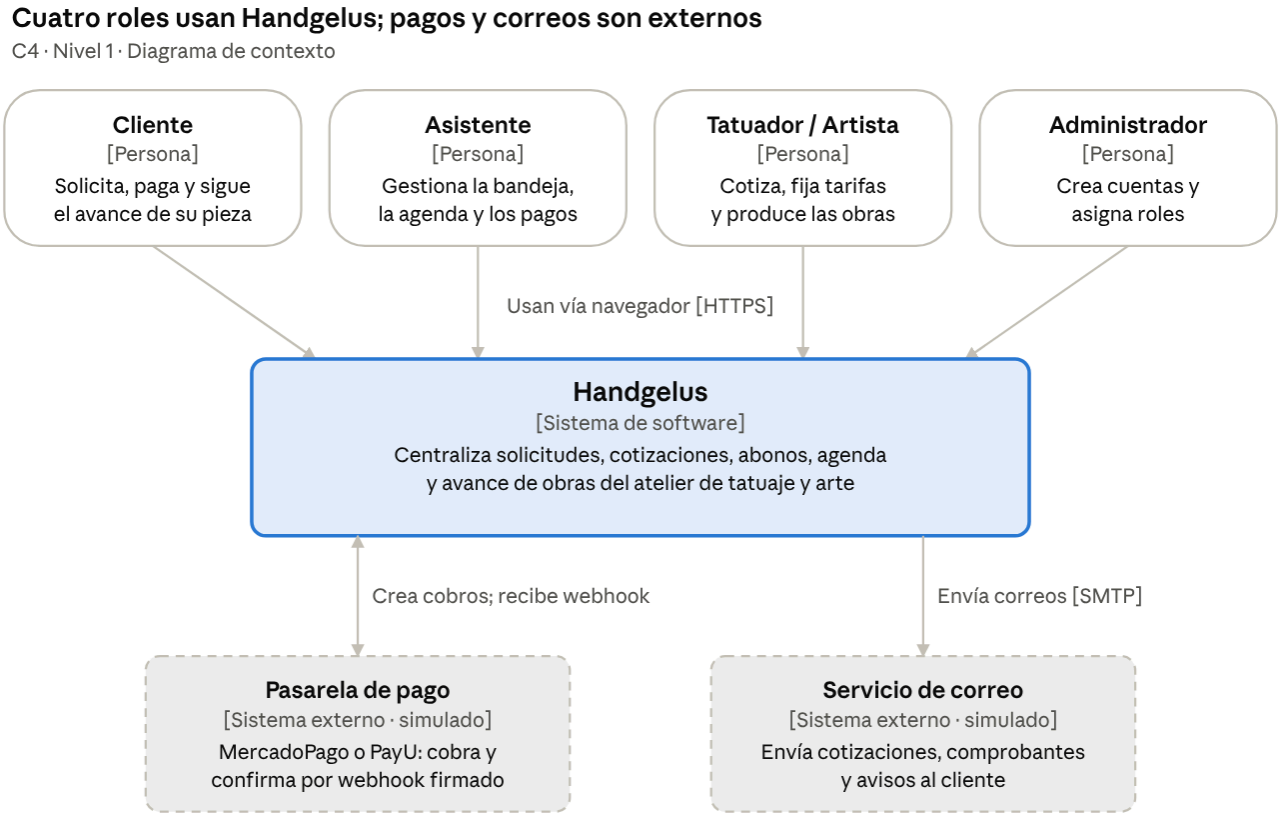
Handgelus es una aplicación de tres contenedores: una SPA en Angular, una API REST en Spring Boot y una base de datos Oracle. La API sigue la **arquitectura hexagonal (puertos y adaptadores)**, de modo que el dominio no depende de la web, de la base de datos ni de los servicios externos.

Patrones arquitectónicos

- **Hexagonal / puertos y adaptadores (Cockburn):** el núcleo (`domain` y `application`) define interfaces de entrada (`*UseCase`) y de salida (`*Port`); la infraestructura las implementa. Todas las dependencias apuntan hacia el dominio.
- **Cliente-servidor con API REST sin estado:** la SPA consume JSON por HTTPS y se autentica con un JWT en cada petición.

- **Arquitectura por capas dentro del hexágono:** adaptadores de entrada → casos de uso → dominio ← puertos de salida ← adaptadores de salida.
- **SPA con componentes y servicios (MVC del lado del cliente):** cada pantalla es un componente; `ApiService` y `AuthService` concentran el acceso a datos y la sesión.

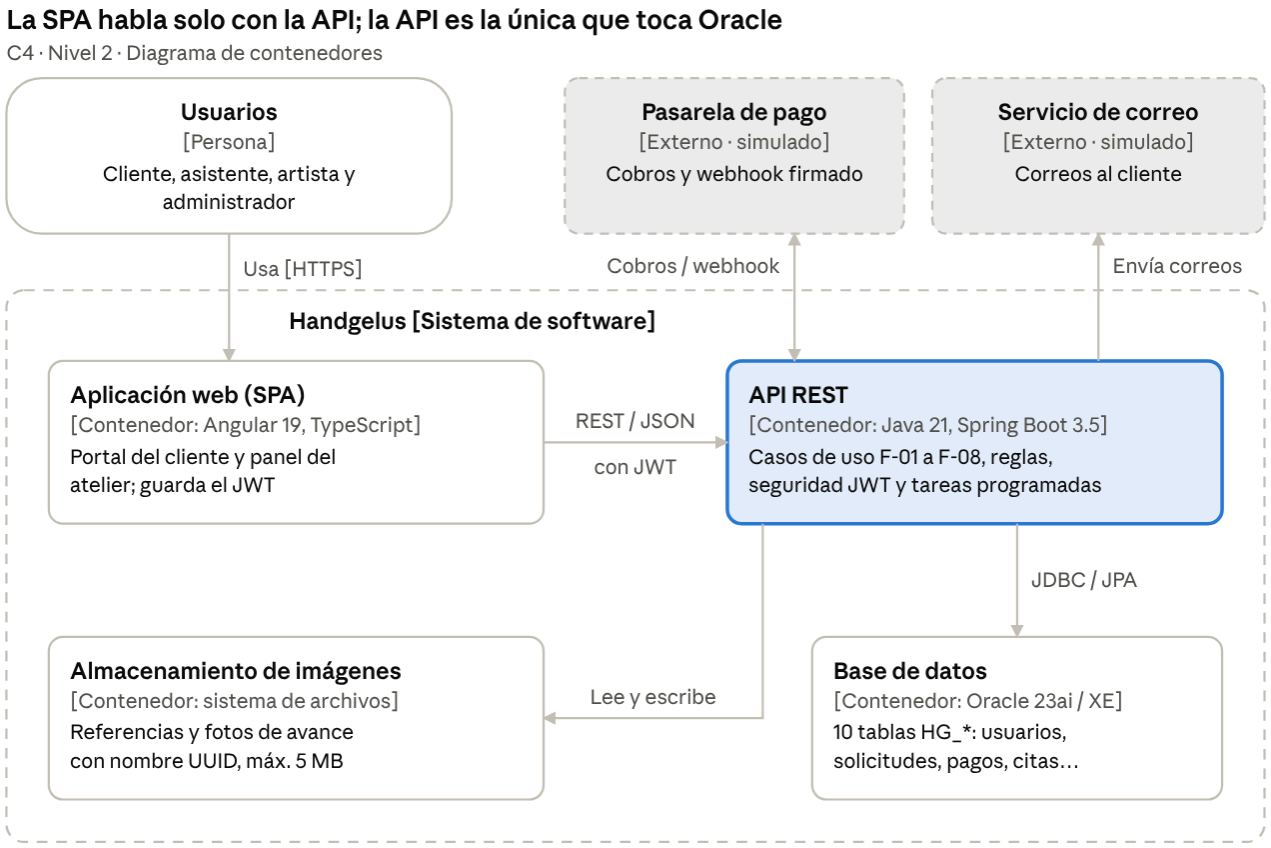
C4 · Nivel 1 · Contexto



C4 nivel 1 · contexto: 4 personas, 1 sistema, 2 sistemas externos

Los cuatro roles usan el mismo sistema desde el navegador. Handgelus delega el cobro a una pasarela externa, que confirma cada pago por *webhook*, y el envío de correos a un servicio externo. En el entorno académico, ambos están simulados.

C4 · Nivel 2 · Contenedores



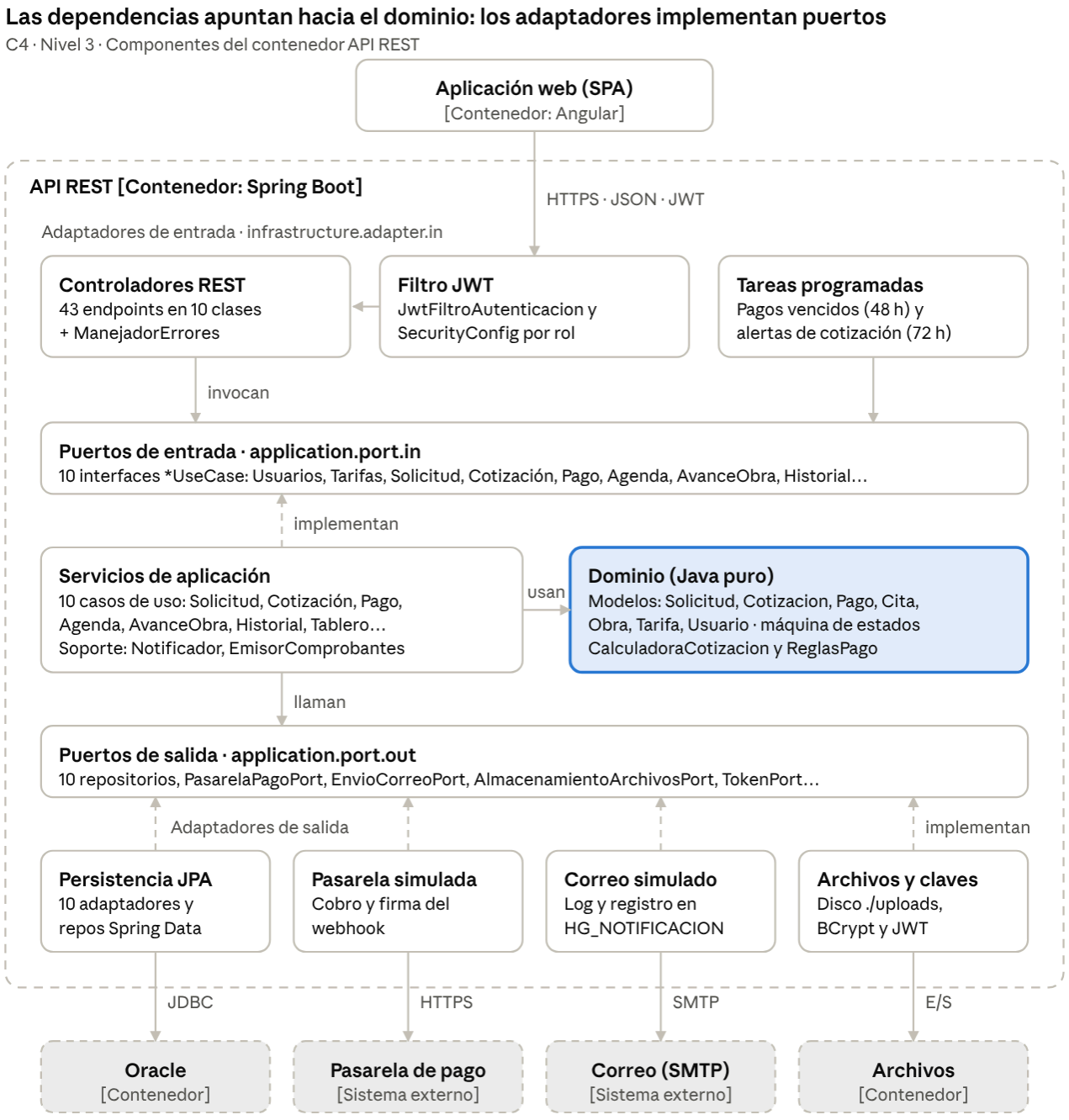
C4 nivel 2 · contenedores: SPA, API, Oracle y almacenamiento de imágenes

La SPA nunca accede a la base de datos: todo pasa por la API, que aplica la autenticación, las reglas y las transacciones. La API guarda los datos en Oracle por JDBC/JPA y las imágenes en el sistema de archivos, y referencia cada imagen por su nombre UUID.

4. Componentes y módulos

La API se divide en tres paquetes con una regla de dependencia estricta: `infrastructure` conoce a `application`, que conoce a `domain`, y nunca al revés. El frontend se organiza por tipo de usuario.

C4 · Nivel 3 · Componentes de la API REST



C4 nivel 3 · componentes de la API REST por capa del hexágono

Las flechas punteadas indican “implementa”: los servicios implementan los puertos de entrada y los adaptadores implementan los puertos de salida. Por eso el dominio (resaltado) no depende de nada externo y se puede probar solo.

Responsabilidades del backend

Componente	Paquete	Responsabilidad
Modelos de dominio	`domain.model`	`Usuario`, `Tarifa`, `Solicitud`, `Cotizacion`, `Pago`, `Cita`, `Obra`, `RegistroAvance`, `Comprobante`, `Notificacion`; cada uno protege sus invariantes y transiciones
Servicios de dominio	`domain.service`	`CalculadoraCotizacion` (RF-04, RN-03, RN-04, RN-09) y `ReglasPago` (30 % / 50 %, saldo, estado del pago)
Excepciones de dominio	`domain.exception`	`ReglaNegocioException` (con código de regla), `AccesoDenegadoException`, `RecursoNoEncontradoException`, `CredencialesInvalidasException`
Puertos de entrada	`application.port.in`	10 interfaces de caso de uso, sus comandos y las vistas de lectura (`SolicitudDetalle`, `HistorialCliente`, `TableroVista`…)
Servicios de aplicación	`application.service`	Orquestan cada caso de uso: validan el rol, cargan agregados, aplican reglas, guardan y notifican
Soporte de aplicación	`application.service.soporte`	`Notificador` (correo y alertas), `EmisorComprobantes` (código HG-AAAA-NNNNN) y `EnsambladorDetalle` (vistas de lectura en lote)
Puertos de salida	`application.port.out`	10 repositorios, `PasarelaPagoPort`, `EnvioCorreoPort`, `AlmacenamientoArchivosPort`, `CifradoPasswordPort`, `TokenPort`
Adaptadores web	`infrastructure.adapter.in.web`	10 controladores REST, `ActorArgumentResolver` (usuario del token) y `ManejadorErrores`
Tareas programadas	`infrastructure.adapter.in.programador`	Cada 10 minutos expiran los pagos de más de 48 h y alertan las cotizaciones de más de 72 h
Persistencia	`infrastructure.adapter.out.persistence`	10 entidades JPA `HG_*`, 10 repositorios Spring Data y 10 adaptadores que traducen entre entidad y dominio
Integraciones	`infrastructure.adapter.out.*`	Pasarela simulada, correo simulado, almacenamiento local, BCrypt y JWT

Componente	Paquete	Responsabilidad
Configuración	`infrastructure.config`	`BeanConfig` (une puertos y adaptadores, transacciones), `SecurityConfig`, `HandgelusProperties`, `DatosIniciales`

Módulos del frontend

Módulo	Contenido	Usuarios
`core`	`ApiService` (un método por endpoint), `AuthService` (sesión con *signals*), `authInterceptor`, *guards* por rol, modelos y etiquetas	Todos
`shared`	*Pipes* (`cop`, `fecha`, `etiqueta`, `imagenSegura`), línea de tiempo, comprobante imprimible, sellos de estado, ilustración botánica	Todos
`layout`	`PublicoLayout` (navegación superior) y `EstudioLayout` (barra lateral)	Todos
`pages/publico`	Inicio, ingresar y registro	Visitantes
`pages/cliente`	Mis proyectos, nueva solicitud, detalle del proyecto, perfil y pagos, pasarela	Cliente
`pages/estudio`	Tablero, bandeja, ficha de solicitud, agenda, proyectos, pagos, clientes, historial, tarifas, usuarios y notificaciones	Asistente, artista y administrador

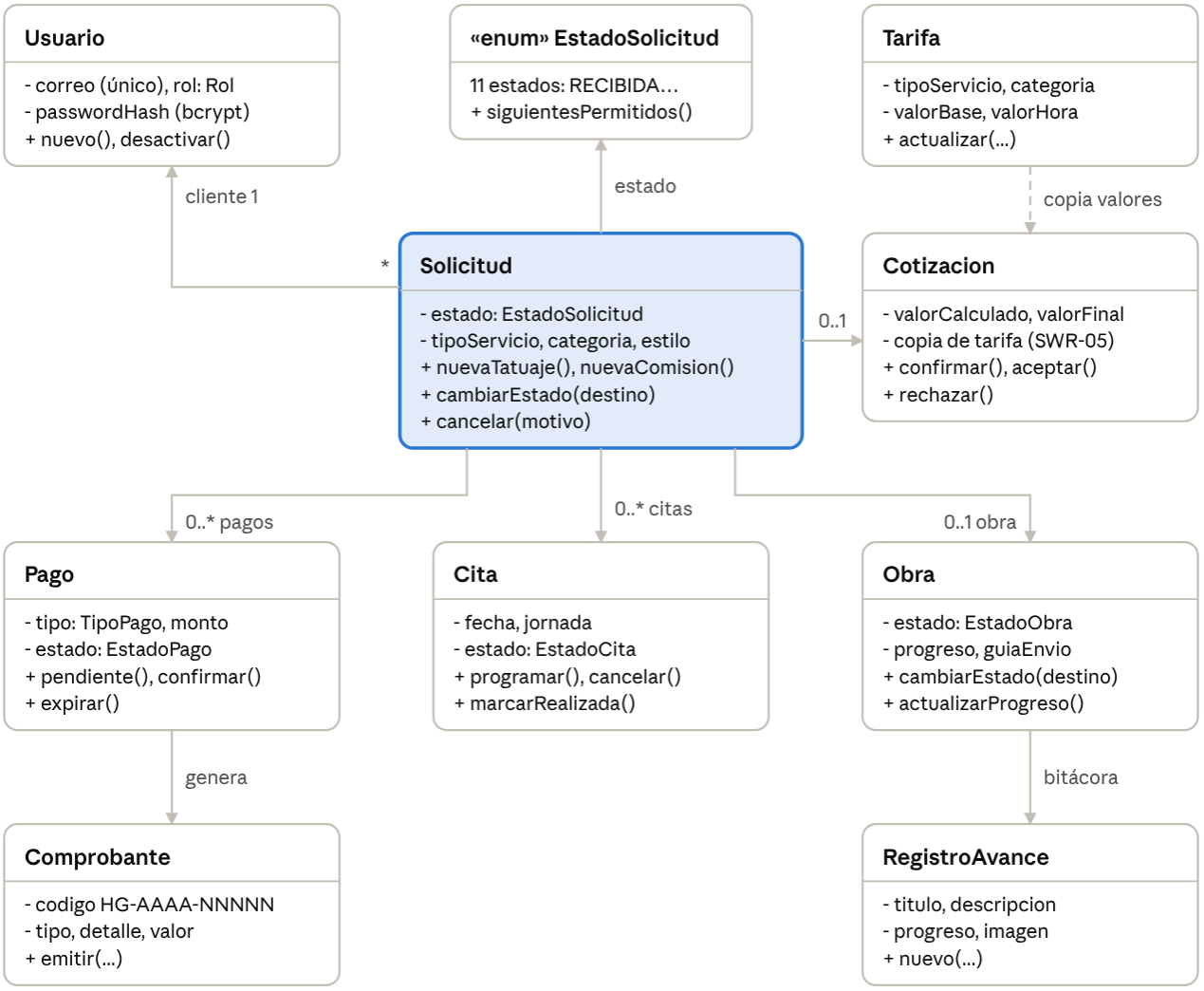
Relaciones entre módulos

- Cada pantalla de Angular llama solo a `ApiService`, y cada método de `ApiService` corresponde a un *endpoint* de un controlador.
- Cada controlador depende de una o dos interfaces `*UseCase`, nunca de un servicio concreto ni de un repositorio.
- Los servicios dependen de puertos de salida y de las clases de soporte; `BeanConfig` les inyecta los adaptadores concretos.
- Los adaptadores de persistencia dependen de los repositorios Spring Data y de las entidades JPA; el dominio no conoce ninguno de los dos.

5. C4 · Nivel 4 · Código

El nivel de código muestra el modelo de dominio (`com.handgelus.domain.model`). Cada solicitud agrupa su cotización, sus pagos, sus citas (si es tatuaje) o su obra (si es comisión), y su bitácora de avances.

La Solicitud es la raíz: cotización, pagos, citas y obra cuelgan de ella
C4 · Nivel 4 · Código del dominio (diagrama de clases)



C4 nivel 4 · modelo de dominio: 10 clases y sus relaciones

- **Solicitud** es la raíz del flujo. Se crea solo mediante `nuevaTatuaje()` o `nuevaComision()`, que validan los campos (SWR-07, SWR-08), y cambia de estado únicamente con `cambiarEstado()`, que consulta `EstadoSolicitud.siguientesPermitidos()`.
- **Cotizacion** guarda la copia de la tarifa con la que se calculó (valor base, valor hora, horas, factor de estilo, envío y sesiones). Por eso un cambio posterior en **Tarifa** no la altera (SWR-05).

- **Pago**, **Cita** y **Obra** tienen su propio ciclo de estados: `PENDIENTE → CONFIRMADO | RECHAZADO | EXPIRADO`, `PROGRAMADA → REALIZADA | CANCELADA` y `EN_ESPERA → EN_PRODUCCION → TERMINADA → ENVIADA`.
- **Comprobante** se emite al aceptar la cotización, al confirmar cada pago y al agendar cada cita (RF-11). El diagrama muestra solo la relación con Pago para no recargarlo.
- Las referencias entre clases son identificadores (`clienteId`, `solicitudId`), no objetos anidados. Así cada clase se guarda y se carga de forma independiente.

Ejemplo: del puerto al adaptador en F-05

Capa	Clase o interfaz	Rol en el pago del abono
Adaptador de entrada	`PagoController`	Recibe `POST /api/solicitudes/{id}/pagos` y el *webhook* `POST /api/pagos/webhook`
Puerto de entrada	`PagoUseCase`	Contrato: `iniciarPago`, `procesarRespuestaPasarela`, `registrarPagoManual`, `expirarPagosVencidos`
Servicio de aplicación	`PagoService`	Orquesta: valida con `ReglasPago`, crea la sesión, cambia el estado y emite el comprobante
Dominio	`Pago`, `ReglasPago`, `Solicitud`	`Pago.pendiente()`, `validarPagoInicial()` y `cambiarEstado(PAGO_PARCIAL_CONFIRMADO)`
Puertos de salida	`PagoRepositoryPort`, `PasarelaPagoPort`	Persistir el pago y hablar con la pasarela
Adaptadores de salida	`PagoPersistenceAdapter` → `PagoJpaRepository` → `HG_PAGO`; `PasarelaPagoSimuladaAdapter`	Implementaciones concretas, inyectadas por `BeanConfig`

6. Interacciones y flujos de datos (C4+1 · diagramas de secuencia)

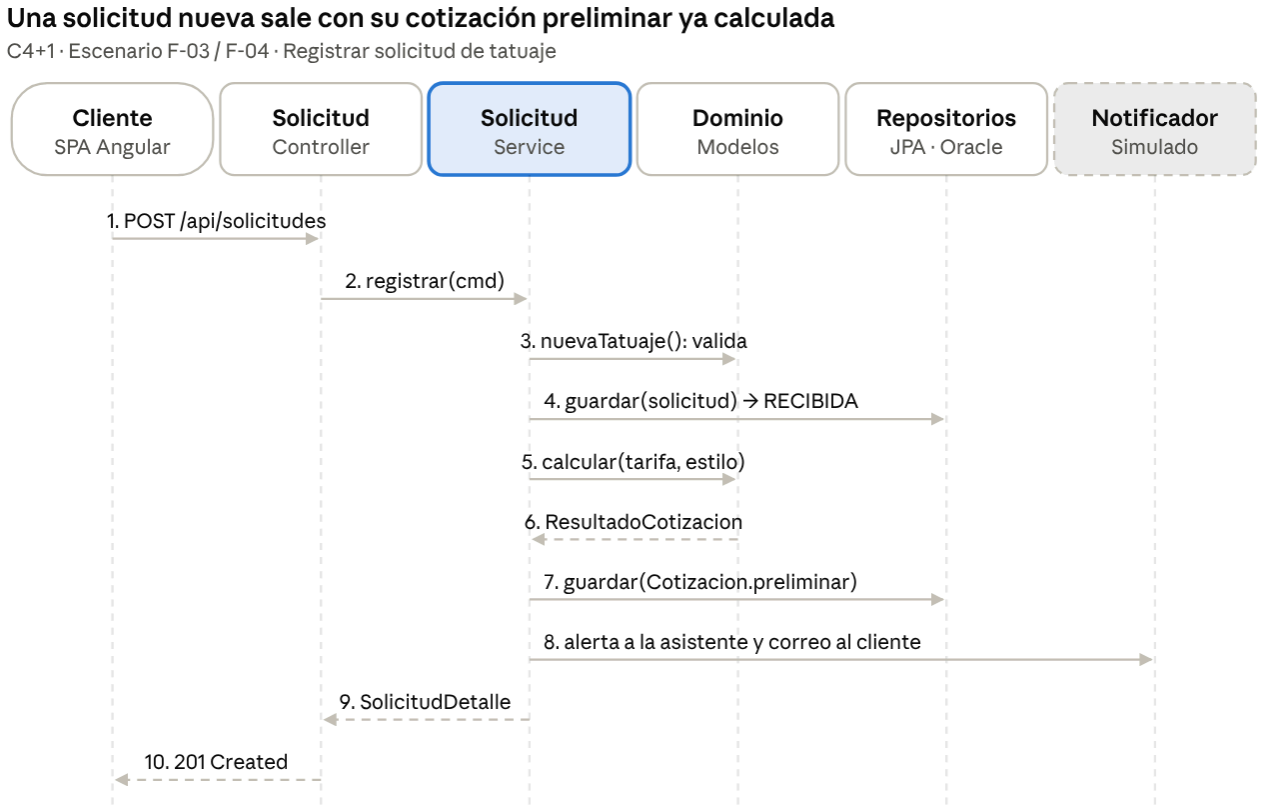
Toda interacción sigue el mismo camino: SPA → controlador → caso de uso → dominio y puertos de salida → respuesta. Cada caso de uso corre en una sola transacción: si una regla falla, nada se guarda.

Flujo de datos general

1. La SPA envía JSON (o `multipart/form-data` cuando hay imagen) con el encabezado `Authorization: Bearer <JWT>`.
2. `JwtFiltroAutenticacion` valida el token y la cuenta activa; `ActorArgumentResolver` convierte el usuario del token en un `Actor` (id, nombre, rol).

3. El controlador valida el formato (`@Valid`) y arma un comando (`RegistrarSolicitudComando` , `AgendarCitaComando` ...).
4. El servicio verifica el rol y la pertenencia del recurso, carga los modelos por los puertos, aplica las reglas del dominio y guarda.
5. Los efectos externos (correo, comprobante, pasarela) se piden por sus puertos. La respuesta es una vista de lectura (`SolicitudDetalle`) que la SPA pinta tal cual.
6. Si se lanza una `ReglaNegocioException` , `ManejadorErrores` responde 422 con el mensaje y el código de la regla, y la SPA lo muestra en un aviso.

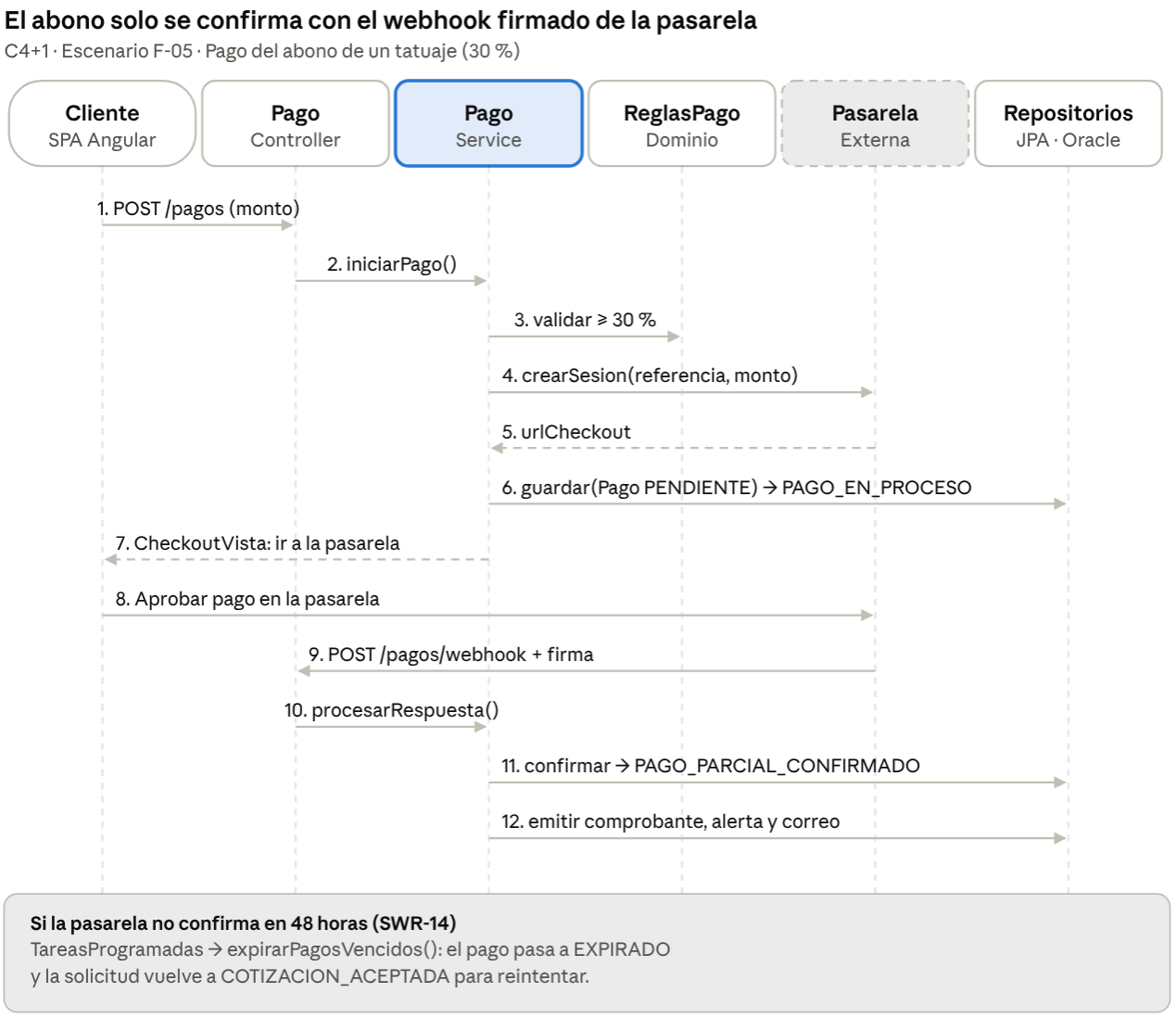
Escenario 1 · Registrar solicitud y cotización automática (F-03, F-04)



C4+1 · secuencia F-03 / F-04 · registrar solicitud

Al recibir la solicitud, el sistema calcula de inmediato el valor preliminar con la tabla de tarifas y guarda la copia de la tarifa en la cotización. Luego el tatuador la revisa y la confirma con `POST /cotizacion/confirmar` , y el cliente recibe el correo.

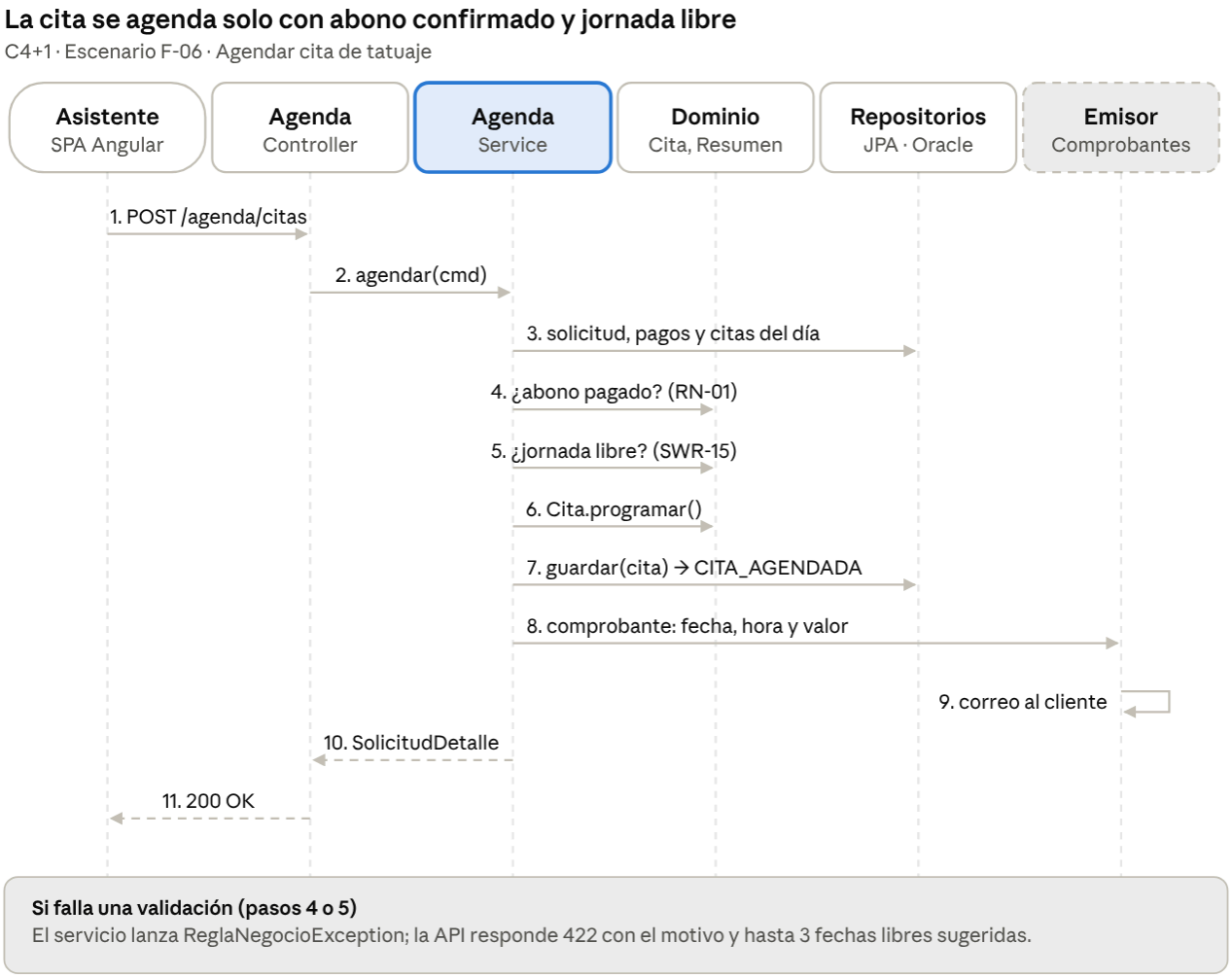
Escenario 2 · Pago del abono por la pasarela (F-05)



C4+1 · secuencia F-05 · pago del abono con webhook

El servicio nunca da por pagado un abono porque la SPA lo diga: solo el *webhook* firmado de la pasarela lo confirma. Si la pasarela reenvía el mismo *webhook*, el pago ya confirmado no se procesa otra vez.

Escenario 3 · Agendar cita de tatuaje (F-06)



C4+1 · secuencia F-06 · agendar cita

La agenda valida tres condiciones antes de crear la cita: abono confirmado (RN-01), domingo cerrado y jornada libre con máximo 2 citas por día (SWR-15, RN-05). El comprobante de la cita lleva fecha, jornada y valor total, y se envía al cliente en la misma transacción.

7. Decisiones de diseño y justificaciones

La decisión central fue aislar las reglas del negocio del framework y de la base de datos. Las demás decisiones se derivan de esa.

#	Decisión	Justificación	Alternativa descartada
D1	Arquitectura hexagonal con `domain` y `application` en Java puro, sin anotaciones de Spring ni JPA	Las reglas (30 %, 48 h, 2 citas/día) se prueban sin Spring ni Oracle, y cambiar la pasarela o el correo no toca el núcleo	Capas clásicas controller/service/repository con entidades JPA en el dominio
D2	Entidades JPA separadas de los modelos de dominio, con métodos `desde()` y `aDominio()`	El modelo de dominio protege sus invariantes; la tabla puede cambiar sin tocar las reglas	Anotar el dominio con `@Entity` (acopla el dominio a JPA)
D3	La cotización guarda una copia de la tarifa usada (valor base, valor hora, horas, factor, envío)	Cumple SWR-05: editar la tabla de tarifas no altera las cotizaciones emitidas	Calcular siempre desde la tarifa vigente
D4	La máquina de estados vive en el enum `EstadoSolicitud` (`siguientesPermitidos()`)	Una sola fuente de verdad para las transiciones del diagrama DE-01; los casos de uso no pueden saltarse etapas	Validar el estado con condicionales repartidos por los servicios
D5	Transacciones aplicadas por AOP desde `BeanConfig` a todo `application.service`	La capa de aplicación queda sin `@Transactional` y cada caso de uso es atómico	`@Transactional` en cada servicio
D6	Autenticación JWT sin estado y validación de rol en dos niveles (URL y caso de uso)	Escala horizontalmente sin sesiones compartidas; la defensa en profundidad evita fugas si se olvida una regla de URL	Sesiones HTTP con cookie

#	Decisión	Justificación	Alternativa descartada
D7	Pasarela y correo como puertos con adaptadores simulados	El curso no dispone de cuentas reales; el contrato del puerto permite conectar MercadoPago, PayU o SMTP sin cambiar el dominio	Codificar la llamada a un proveedor concreto
D8	El tiempo se inyecta como `Clock`	Las reglas de 48 h y 72 h se prueban adelantando un reloj, sin esperas reales	`LocalDateTime.now()` disperso en el código
D9	Parámetros de negocio en `application.properties` (`handgelus.negocio.*`)	El propietario cambia porcentajes, plazos o costos de envío sin recompilar	Constantes en el código
D10	Frontend Angular con componentes *standalone*, *signals* y rutas con carga diferida	Menos código de módulos, reactividad fina y un paquete inicial pequeño	NgModules y carga completa

Atributos de calidad

- **Rendimiento:** las vistas de lista consultan en lote, con `findAllById` y consultas `IN`, para evitar el problema N+1. Hay índices en `HG_SOLICITUD(cliente_id)`, `HG_SOLICITUD(estado)`, `HG_CITA(tatuador_id, fecha)` y `HG_PAGO(solicitud_id)`. El *pool* Hikari tiene 10 conexiones y `open-in-view` está desactivado. El frontend usa carga diferida por pantalla.
- **Escalabilidad:** el backend no guarda estado de sesión (JWT), así que puede replicarse detrás de un balanceador. Para varias instancias habría que mover las imágenes de `./uploads` a un almacenamiento compartido (otro adaptador de `AlmacenamientoArchivosPort`) y coordinar las tareas programadas para que corran en una sola instancia.

- **Mantenibilidad:** hay un caso de uso por funcionalidad (`F-01` a `F-08`) y cada puerto de salida tiene una sola implementación intercambiable. Los nombres del código están en el lenguaje del negocio (`Solicitud` , `Cotizacion` , `Abono`) y los comentarios citan el requisito que implementan (`SWR-14` , `RN-07`).
- **Usabilidad:** los mensajes de error están en español claro (RNF-11). La solicitud se completa en tres bloques (RNF-01) y la interfaz es *responsive*, con foco visible y movimiento reducido cuando el usuario lo prefiere.

8. Patrones de diseño utilizados

El sistema combina patrones arquitectónicos (hexagonal, MVC en el cliente) con patrones de diseño clásicos y tácticos de DDD.

Patrón	Tipo	Dónde se aplica en Handgelus
Puertos y adaptadores (hexagonal)	Arquitectónico	`application.port.in` (casos de uso) y `port.out` (repositorios, pasarela, correo); los adaptadores viven en `infrastructure.adapter`
Repository	Persistencia	`SolicitudRepositoryPort` , implementado por `SolicitudPersistenceAdapter` sobre `SolicitudJpaRepository`
Adapter	Estructural	`PasarelaPagoSimuladaAdapter` adapta la pasarela externa a `PasarelaPagoPort` ; `BcryptCifradoAdapter` adapta el `PasswordEncoder` de Spring a `CifradoPasswordPort`
Data Mapper	Persistencia	`SolicitudEntity.desde(solicitud)` y `entity.aDominio()` traducen entre la tabla y el modelo
State	Comportamiento	`EstadoSolicitud.siguientesPermitidos()` y `Solicitud.cambiarEstado()` aplican la máquina de estados; `Obra.cambiarEstado()` exige el orden en espera → producción → terminada → enviada
Strategy	Comportamiento	`Estilo` aporta el factor de complejidad que cambia el cálculo; cada implementación de `PasarelaPagoPort` o `EnvioCorreoPort` es una estrategia intercambiable

Patrón	Tipo	Dónde se aplica en Handgelus
Factory Method	Creacional	`Solicitud.nuevaTatuaje(...)` y `Solicitud.nuevaComision(...)`, `Pago.pendiente(...)` y `Pago.confirmadoManual(...)`, `Cotizacion.preliminar(...)`: construyen objetos válidos o lanzan `ReglaNegocioException`
Domain Service	DDD	`CalculadoraCotizacion` y `ReglasPago`: reglas que no pertenecen a una sola entidad
Value Object / DTO	DDD / Integración	Records inmutables: `ResultadoCotizacion`, `ParametrosNegocio`, `Actor`, `SolicitudDetalle`; comandos como `RegistrarSolicitudComando`
Command	Comportamiento	Cada caso de uso recibe un *record* comando (`AgendarCitaComando`, `TarifaComando`) que encapsula la petición
Facade	Estructural	`EnsambladorDetalle` reúne siete repositorios en una vista de lectura (`SolicitudDetalle`)
Dependency Injection / Composition Root	Creacional	`BeanConfig` es el único punto que conoce a la vez los servicios y los adaptadores, y los une por constructor
Proxy (AOP)	Estructural	El `Advisor` transaccional envuelve los servicios de aplicación sin modificarlos
Chain of Responsibility	Comportamiento	La cadena de filtros de Spring Security con `JwtFiltroAutenticacion`; el `authInterceptor` de Angular
Observer	Comportamiento	*Signals* y `Observable` de RxJS en el frontend; las pantallas reaccionan a los cambios de sesión y de datos
Template / Guard	Frontend	`rolGuard`, `sesionGuard` e `invitadoGuard` protegen las rutas por rol

Ejemplo: Factory Method con validación de invariantes

```
// domain/model/Solicitud.java
public static Solicitud nuevaComision(Long clienteId, String titulo, String
idea,
        CategoriaTamano formato, Estilo estilo, String direccion, String
ciudad,
        String imagen, LocalDateTime ahora) {
    validarComunes(clienteId, titulo, idea, formato);
    if (esVacio(direccion) || direccion.trim().length() < 8 ||
esVacio(ciudad)) {
        throw new ReglaNegocioException("SWR-08",
            "Debe indicar la dirección completa de destino (dirección y
ciudad)...");
    }
    return new Solicitud(null, clienteId, TipoServicio.COMISION, ...,
EstadoSolicitud.RECIBIDA, ...);
}
```

Ejemplo: State en la máquina de estados

```
// domain/model/EstadoSolicitud.java
public Set<EstadoSolicitud> siguientesPermitidos() {
    return switch (this) {
        case RECIBIDA -> EnumSet.of(COTIZACION_GENERADA, CANCELADA);
        case COTIZACION_ACEPTADA -> EnumSet.of(PAGO_EN_PROCESO,
PAGO_PARCIAL_CONFIRMADO, CANCELADA);
        case PAGO_EN_PROCESO -> EnumSet.of(PAGO_PARCIAL_CONFIRMADO,
COTIZACION_ACEPTADA, CANCELADA);
        case TERMINADA -> EnumSet.of(ENVIADA);
        // ...
    };
}
```

Ejemplo: Adapter detrás de un puerto

```
// application/port/out/PasarelaPagoPort.java (núcleo)
public interface PasarelaPagoPort {
    SesionPago crearSesion(String referencia, BigDecimal monto, String
concepto, String correoCliente);
    boolean firmaValida(String firma);
}

// infrastructure/adapter/out/pasarela/PasarelaPagoSimuladaAdapter.java
@Component
public class PasarelaPagoSimuladaAdapter implements PasarelaPagoPort { ... }
```

9. Gestión de la calidad y pruebas

La prueba principal ejecuta las 8 funcionalidades de punta a punta sobre los casos de uso reales y pasó 61 de 61 verificaciones. Como el núcleo hexagonal no depende de Spring, la prueba usa adaptadores en memoria en lugar de Oracle.

Estrategia de prueba

1. **Pruebas del núcleo (unitarias e integradas en memoria):** `FlujoNegocioTest` instancia los servicios reales con repositorios en memoria, una pasarela y un correo falsos, y un reloj controlable (`MutableClock`). El reloj permite comprobar las reglas de 48 h y 72 h sin esperar.
2. **Validación en cada capa:** las anotaciones `@Valid` y `@NotBlank` en los DTO REST, las invariantes en las fábricas del dominio y las restricciones de base de datos (`UNIQUE` en correo, referencia de pago, código de comprobante y tarifa por tipo y tamaño).
3. **Verificación del frontend:** comprobación de tipos con TypeScript estricto (`strict`, `noImplicitReturns`) y revisión de que cada plantilla importe sus *pipes* y componentes. También se renderizaron la portada y el tablero para revisar el diseño.
4. **Pruebas de aceptación manuales:** el recorrido cliente → artista → cliente → asistente con las cuentas de demostración, siguiendo los criterios de aceptación de los PBI-01 a PBI-08.

Casos verificados por `FlujoNegocioTest`

Funcionalidad	Casos (verificaciones)	Ejemplos de lo que se comprueba
F-01	6	Rol único; la asistente no crea cuentas; correo duplicado rechazado; la contraseña no se guarda en texto plano
F-02	2	Solo el tatuador o el administrador editan tarifas
F-03	9	Imagen de 6 MB rechazada; comisión sin dirección rechazada; estado inicial Recibida; cálculo de $1.342.000; 2 sesiones (RN-04)
F-04	7	SWR-05: la cotización conserva su valor tras cambiar la tarifa; solo el tatuador confirma; correo al cliente; otro cliente no ve la solicitud
F-05	8	Abono menor al 30 % rechazado; webhook con firma inválida rechazado; expiración a las 49 h y vuelta a Cotización aceptada; webhook idempotente
F-06	8	Sin abono no hay cita; jornada ocupada con fechas sugeridas; tercera cita del día rechazada; domingo cerrado; RN-07 no cierra con saldo
F-07	11	Anticipo del 50 % ($353.500); no salta de En espera a Terminada; no se envía con saldo pendiente; envío con pago completo
F-08	6	Historial con 2 servicios y $2.007.000 pagados; cliente ajeno rechazado; búsqueda sin tildes
Transversal	4	Alerta de cotización sin confirmar a las 72 h; rechazo → Cancelada; ingresos del tablero; bandeja Finalizadas
Total	**61**	**61 correctas, 0 fallidas**

Métricas del código

Módulo	Archivos	Líneas
`domain` (modelos, reglas, excepciones)	35	1.448

Módulo	Archivos	Líneas
`application` (puertos y casos de uso)	52	2.485
`infrastructure` (adaptadores y configuración)	55	3.282
Pruebas (`src/test`)	1	317
Frontend Angular (`src`)	40	5.043

El núcleo (`domain` y `application`) suma 3.933 líneas sin ninguna dependencia de Spring ni de JPA. Compila con `javac` puro, lo que confirma que la regla de dependencia de la arquitectura hexagonal se cumple. El API expone 43 *endpoints* REST.

Pendiente: aún no hay pruebas de los adaptadores (controladores con `MockMvc` y repositorios con Oracle mediante Testcontainers), ni pruebas de componentes Angular o de extremo a extremo con Playwright. Tampoco se ha medido la cobertura con JaCoCo.

10. Despliegue y configuración

El sistema se despliega como tres procesos: el servidor de Angular (puerto 4200), el backend Spring Boot (8080) y Oracle (1521). En desarrollo, el servidor de Angular redirige `/api` al backend mediante `proxy.conf.json` , por lo que el navegador ve un solo origen.

Entorno de implementación

Nodo	Tecnología	Puerto	Configuración
Navegador	Chrome, Edge, Firefox	—	Guarda el JWT en `localStorage`
Servidor del frontend	`ng serve` (desarrollo) o `ng build` → archivos estáticos en Nginx o Apache (producción)	4200 / 80	`proxy.conf.json`
Backend	JAR de Spring Boot sobre JDK 21	8080 (8443 con HTTPS)	`application.properties` , `application-h2.properties`
Base de datos	Oracle XE 21c (`XEPDB1`) u Oracle 23ai Free en Docker (`FREEPDB1`)	1521	Usuario `HANDGELUS` ; tablas creadas por Hibernate (`ddl-auto=update`)

Nodo	Tecnología	Puerto	Configuración
Almacenamiento	Sistema de archivos del servidor	—	`handgelus.almacenamiento.directorio=./uploads`

Configuración por *properties*

Grupo	Propiedades	Valor por defecto
Conexión	`spring.datasource.url`, `username`, `password` (variables `DB_URL`, `DB_USUARIO`, `DB_CLAVE`)	`jdbc:oracle:thin:@localhost:1521/XEPDB1`, `HANDGELUS`
Seguridad	`handgelus.jwt.secreto`, `jwt.expiracion-horas`, `seguridad.requerir-https`, `cors.origenes-permitidos`	12 h, HTTPS desactivado, `http://localhost:4200`
Negocio	`handgelus.negocio.porcentaje-abono-tatuaje`, `porcentaje-anticipo-comision`, `horas-expiracion-pago`, `horas-alerta-cotizacion`, `max-citas-por-dia`, `imagen-max-bytes`, `costo-envio-local`, `costo-envio-nacional`	30, 50, 48, 72, 2, 5 MB, $12.000, $28.000
Tareas	`handgelus.tareas.intervalo-ms`	600.000 (10 min)
Datos de demostración	`handgelus.datos-iniciales.habilitado`	`true`

Proceso de despliegue

1. **Base de datos:** levantar Oracle con `docker compose up -d` o ejecutar `db/oracle-crear-usuario.sql` como SYSTEM.
2. **Backend:** `./gradlew :backend:build` genera `backend/build/libs/backend-1.0.0.jar`. En desarrollo basta `./gradlew :backend:bootRun`; en el servidor se ejecuta `java -jar backend-1.0.0.jar` con las variables `DB_URL`, `DB_USUARIO` y `DB_CLAVE`, y un `handgelus.jwt.secreto` propio.
3. **Frontend:** `npm install` y `npm run build` producen `frontend/dist/handgelus`, que se publica en el servidor web con una regla que envíe `/api` al backend.
4. **Verificación:** al arrancar, el registro muestra `Started HandgelusApplication` y Hibernate crea o actualiza las tablas `HG_*`.

Actualización

- **Cambios de código:** se reconstruyen y reemplazan el JAR y la carpeta `dist`. Como el backend no guarda sesiones, puede reiniciarse sin perder los ingresos de los usuarios mientras el token siga vigente.
- **Cambios de esquema:** `ddl-auto=update` agrega columnas y tablas nuevas, pero no elimina ni renombra. Para producción se recomienda pasar a `validate` y versionar el esquema con Flyway o Liquibase.
- **Cambios de reglas:** porcentajes, plazos y costos se ajustan en `application.properties` y se aplican con un reinicio; las tarifas se cambian en caliente desde la pantalla Tarifas.
- **Desarrollo en IntelliJ:** la carpeta `.run/` trae tres configuraciones listas: *Backend - Oracle*, *Backend - H2 (sin Oracle)* y *Frontend - Angular*.

11. Consideraciones de seguridad

Cada petición se valida dos veces: Spring Security filtra por URL y rol, y luego cada caso de uso vuelve a comprobar el rol y que el recurso pertenezca al usuario.

Medidas implementadas

Amenaza	Medida	Dónde
Robo de contraseñas	BCrypt con factor 12; nunca se devuelve el *hash* (`UsuarioVista` no lo incluye)	`SecurityConfig`, `BcryptCifradoAdapter`
Suplantación de sesión	JWT firmado con HMAC-SHA (secreto ≥ 32 caracteres), emisor `handgelus`, vigencia de 12 h; las cuentas desactivadas pierden acceso de inmediato	`JwtTokenAdapter`, `JwtFiltroAutenticacion`
Tráfico sin cifrar (SWR-03)	`handgelus.seguridad.requerir-https=true` exige canal seguro; *keystore* PKCS12 en `server.ssl.*`	`SecurityConfig`
Acceso a datos ajenos (SWR-19)	`Actor.exigirPropietarioOPersonal()`: un cliente solo ve sus solicitudes, historial, comprobantes e imágenes	Casos de uso de solicitud, historial y archivos
Webhook falso de la pasarela	Firma en la cabecera `X-Firma-Pasarela`, comparada en tiempo constante (`MessageDigest.isEqual`); procesamiento idempotente	`PasarelaPagoSimuladaAdapter`, `PagoService`

Amenaza	Medida	Dónde
Archivos maliciosos	Máximo 5 MB; solo PNG, JPG, WEBP o PDF; nombre aleatorio UUID; patrón de nombre validado contra *path traversal*; cabecera `nosniff`	`AlmacenamientoLocalAdapter`, `SolicitudController`
Peticiones de otros sitios	CORS limitado a `handgelus.cors.origenes-permitidos`; API sin estado y sin cookies, por lo que CSRF no aplica	`SecurityConfig`
Inyección SQL	Consultas JPQL parametrizadas y *Criteria API*; sin concatenación de SQL	Repositorios JPA
Fuga de información en errores	`ManejadorErrores` devuelve mensajes de negocio y oculta las trazas internas	Adaptador web
Datos personales (Ley 1581 de 2012)	Autorización expresa del tratamiento de datos al registrarse	`AutenticacionService`

Gestión de roles y permisos

Cada cuenta tiene un rol único (SWR-01). Solo el administrador crea cuentas internas, y nadie puede cambiar su propio rol ni desactivar su propia cuenta.

Acción	Cliente	Asistente	Tatuador / Artista	Administrador
Registrarse y enviar solicitudes	Sí	—	—	—
Aceptar o rechazar su cotización y pagar en línea	Sí (solo las suyas)	—	—	—
Ver su proyecto, avance e historial	Sí (solo los suyos)	Sí	Sí	Sí
Ver bandeja, tablero y notificaciones	—	Sí	Sí	Sí
Confirmar o ajustar el valor de la cotización	—	—	Sí (exclusivo, RF-05)	—
Configurar la tabla de tarifas	—	—	Sí	Sí
Registrar pagos recibidos en el atelier	—	Sí	Sí	Sí

Acción	Cliente	Asistente	Tatuador / Artista	Administrador
Agendar, cancelar o cerrar citas	—	Sí	Sí	Sí
Cambiar el estado de la obra y registrar avances	—	Sí	Sí	Sí
Consultar el historial de cualquier cliente	—	Sí	Sí	Sí
Crear, editar y desactivar cuentas	—	—	—	Sí

En el frontend, `rolGuard` oculta y bloquea las rutas que el rol no puede usar. Es solo una ayuda de navegación: la autorización real la aplica el backend.

12. Referencias y recursos adicionales

Documentos del proyecto

- *Plan vivo de requisitos — Ingeniería de Software II* (`plan-requisitos.xlsx`): trazabilidad ISO/IEC/IEEE 29148, F-01 a F-08, SWR-01 a SWR-21 y PBI en Jira (HAND-2 a HAND-9).
- *Proyecto Handgelus* (`proyecto-handgelus.pdf`): contexto del negocio, reglas RN, requisitos RF y RNF, casos de uso CU-01 a CU-03 y diagrama de estados DE-01.
- `README.md` del repositorio: instrucciones de ejecución, cuentas de demostración y API REST.

Arquitectura y diseño

- Simon Brown, *The C4 model for visualising software architecture*: c4model.com
- Alistair Cockburn, *Hexagonal Architecture (Ports and Adapters)*: alistair.cockburn.us/hexagonal-architecture
- Philippe Kruchten, *The 4+1 View Model of Architecture*, IEEE Software, 1995.
- Eric Evans, *Domain-Driven Design*, Addison-Wesley, 2003.
- Erich Gamma, Richard Helm, Ralph Johnson y John Vlissides, *Design Patterns*, Addison-Wesley, 1994.
- Martin Fowler, *Patterns of Enterprise Application Architecture* (Repository, Data Mapper), Addison-Wesley, 2002.
- ISO/IEC/IEEE 29148:2018, *Requirements engineering*.

Documentación técnica

- Spring Boot: docs.spring.io/spring-boot
- Spring Security: docs.spring.io/spring-security
- Spring Data JPA: docs.spring.io/spring-data/jpa
- Angular: angular.dev
- Oracle Database Free: oracle.com/database/free
- Imagen Docker `gvenzl/oracle-free`: hub.docker.com/r/gvenzl/oracle-free
- JJWT: github.com/jwtk/jjwt
- Gradle: docs.gradle.org
- Ley 1581 de 2012 (protección de datos personales, Colombia): funcionpublica.gov.co

Herramientas

IntelliJ IDEA, Gradle 8.14, Node.js y npm, Angular CLI 19, Docker Desktop, Oracle SQL Developer o el cliente de base de datos de IntelliJ, Jira (gestión del *backlog*) y Git.